SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

*This section introduces **AI Rent Advisor**, a system designed to help users make better rental decisions by providing insights on rental prices, locations, and property conditions. The system addresses common issues such as lack of transparency in rental pricing, difficulty comparing properties, and limited access to reliable rental information.*

1.1. Purpose:

The purpose of this system analysis document is to outline the technical scope and design decisions behind the development of AI Rent Advisor. It explains how the system is structured, how data flows within the system, and how the main features operate.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *The system is designed as a web-based platform with a modular structure. The frontend handles user interaction, while the backend processes data and manages business logic. A centralized database is used to store property listings, user data, and rental insights.*

2. ***Data Flows:***
    a. *Users input search criteria such as location, rental, and monthly income through the application. The request is sent to the backend, where it retrieves relevant property data from the database. The system processes and filters the data, then returns recommended listings and insights to the user interface.*

3. ***Model Process:***
    The system workflow includes:
    - *User searches for rental properties*
    - *System filters and analyzes available listings*
    - *Rental insights (e.g. average price, comparisons) are generated*
    - *Results are displayed to help users make decisions*

4. ***Role of Reference:***
    a. *This document serves as a reference for developers and testers to understand the overall system design, data handling, and functionality of Rent Advisor.*

1.2. ***Background**:*

© UMHackathon 2026

Finding suitable rental properties can be challenging due to scattered information, inconsistent pricing, and lack of reliable comparison tools. Many users rely on multiple platforms, which makes the process time-consuming and inefficient.

AI Rent Advisor aims to centralize rental information and provide meaningful insights to simplify the decision-making process.

1. *Previous Version:*

Currently, rental platforms mainly provide:

- *Property listings with basic details*
- *Search and filtering functions*
- *Limited comparison features*

However, these platforms often lack:

- *Clear price insights*
- *Data-driven recommendations*
- *Transparency in rental trends*

2. *Changes in Major Architectural Components:*

The proposed system introduces:

- *A data-driven recommendation component*
- *Enhanced filtering and comparison features*
- *Integration of rental insights (e.g. average pricing, trends)*
- *Improved user interface for better decision-making*

3. *New capabilities that are only introduced in AI Rent Advisor*

- *Smart rental recommendations based on user preferences*
- *Price comparison across similar properties*
- *Insights into rental trends by location*
- *User-friendly search and filtering system*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Tenant	*Searches for properties, compares options, views insights*	*Accurate listings, easy comparison, clear pricing insights*

© UMHackathon 2026

Landlords	*Provide property listings with details*	*Visibility of listings, correct information display*
Development Team	*Build and maintain the system*	*Clear architecture, scalable design, maintainable code*
QA Team	*Test system functionality*	*Well-defined features, reliable outputs, proper validation*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Cross-Platform App (Flutter for Web & Mobile).*
Architecture	*Microservices-based Cloud Architecture (FastAPI + Node.js).*

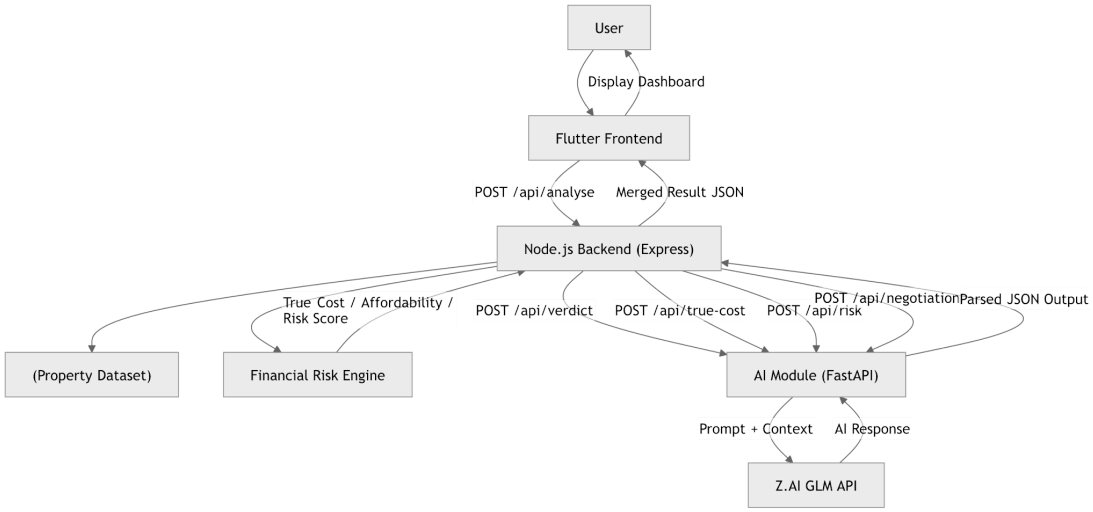
*The **AI Rent Advisor** is a client-server system featuring a **Flutter** frontend that communicates via a **RESTful API** layer. The backend is split into three core microservices: the **Analysis Service** (handling GLM logic), the **Market Service** (handling Kaggle dataset lookups), and the **Risk Service (processing forum data). These are deployed on Vercel and Railway for rapid scalability.***

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

2.1.3 Dependency Diagram

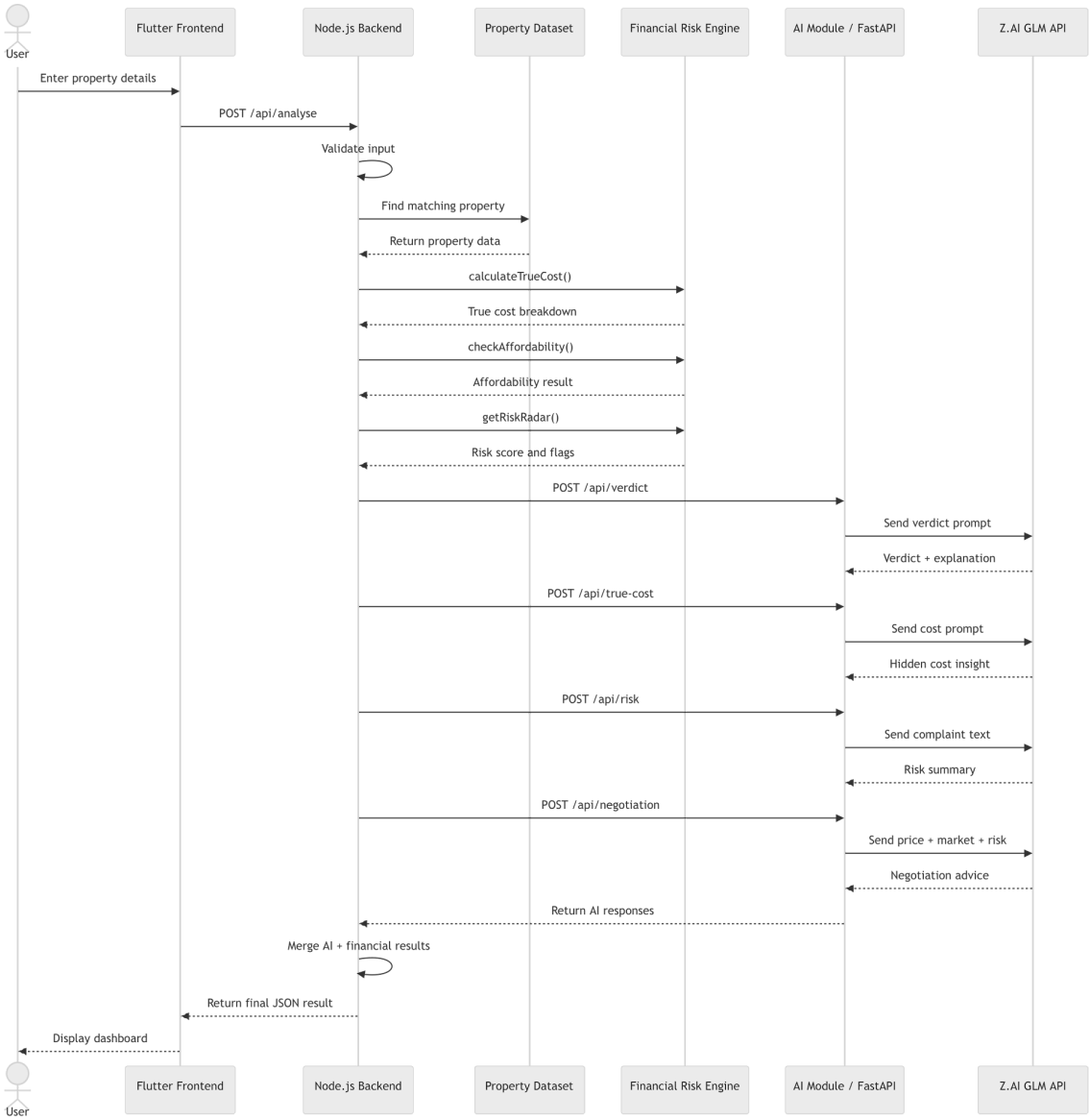
The AI Rent Advisor system uses a distributed architecture where the frontend, backend server, financial risk engine, AI service layer, and dataset interact through API calls. The Flutter frontend sends user input to the Node.js backend. The backend then retrieves property data, performs financial and risk calculations, constructs prompts, and sends the required context to the Z.ai GLM service through the AI module.



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. *Sequence Diagram*



2.2. *Technological Stack*

2.2.1. *Frontend: **Flutter/Dart** — Chosen for its ability to provide high-fidelity UI/UX across both web and mobile platforms from a single codebase, ensuring consistency and faster development.*

2.2.2. *Backend: **Node.js (Express)** — handling API routing, request processing, and integration between system components. **FastAPI (Python)** — AI service layer to interact with the Z.ai GLM model and process AI-driven logic such as rental verdict generation and risk analysis.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

*2.2.3. Authentication Service: **Firebase Authentication** — Used to manage user authentication, including login and session management. It provides secure and scalable identity management without requiring custom backend implementation.*

*2.2.4. Database: **In-memory dataset (mock data)** — Used for storing property listings, rental benchmarks, and complaint data during the hackathon prototype phase. This allows faster development and simplified deployment.*

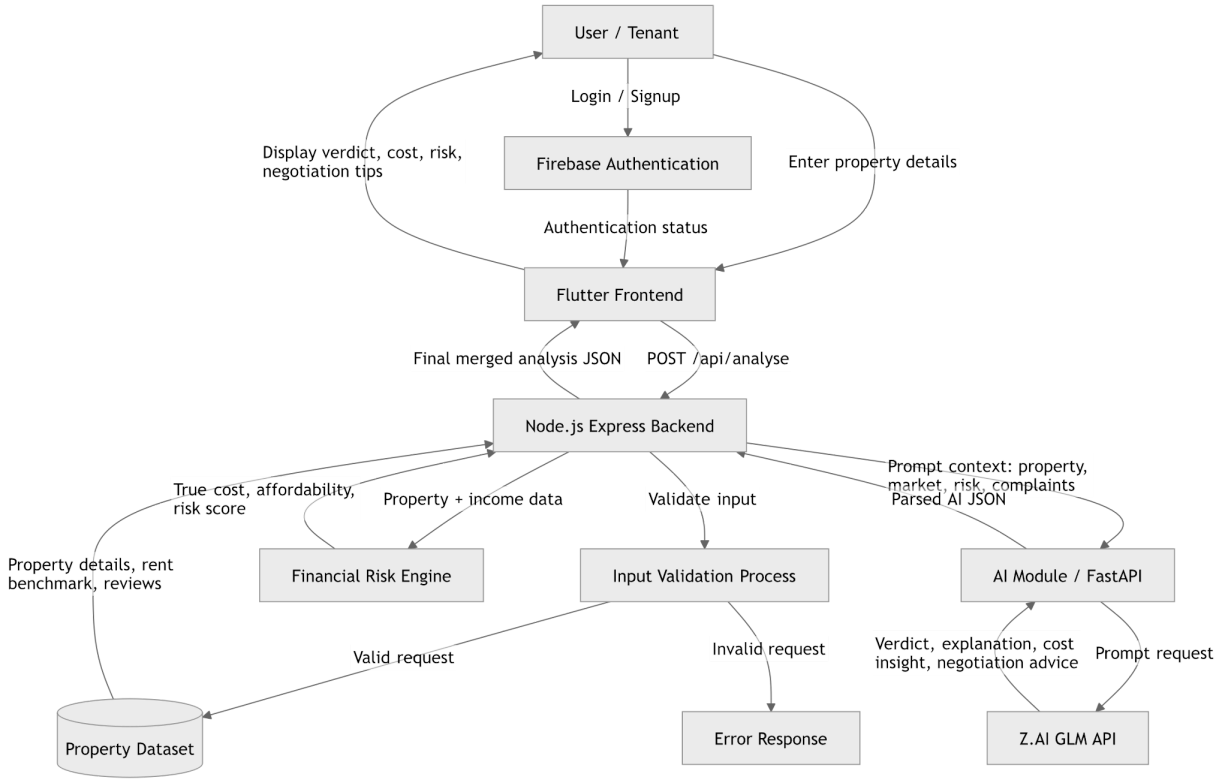
*2.2.5. Deployment: **Vercel** (Frontend) and **Railway** (Backend).*

For future scalability, the system can be extended to integrate a persistent database such as PostgreSQL or Firebase Firestore to support long-term data storage and user-specific history.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



The DFD shows that the user interacts with the system through the Flutter frontend. The frontend sends analysis requests to the backend. The backend acts as the central controller that validates input, retrieves property data, performs financial and risk analysis, and communicates with the AI service.

© UMHackathon 2026

Email: umhackathon@um.edu.my

The Financial Risk Engine calculates deterministic outputs such as affordability ratio, hidden rental costs, and risk score. The AI module sends summarized context to Z.ai GLM for reasoning and explanation generation. Firebase Authentication is used to manage login and user session handling.

*To avoid a **black hole process**, every process in the system has both input and output. For example, the AI module receives prompt context from the backend and returns parsed AI results. The Financial Risk Engine receives property and income data and returns cost and affordability outputs. No process only receives data without producing an output.*

2.3.2. Normalized Database Schema

Since the current hackathon prototype uses an in-memory dataset, the following schema represents the logical database design for a production-ready version. The schema is normalized to 3NF to reduce redundancy and maintain data integrity.

Main Entities

User — stores user information

Field	**Type**	**Description**
user_id	PK	Unique user identifier
name	VARCHAR	Full name
email	VARCHAR	Login email
monthly_income	DECIMAL	Used for affordability analysis
preferred_location	VARCHAR	Default search area

© UMHackathon 2026
Email: umhackathon@um.edu.my

subscription_tier	ENUM	Free / Premium / SaaS
created_at	TIMESTAMP	Account creation date

Property — stores rental listing details

Field	Type	Description
property_id	PK	Unique property identifier
name	VARCHAR	Property/condo name
postcode	FK	Links to Benchmark table
listed_rent	DECIMAL	Advertised monthly rent
room_type	VARCHAR	Studio / 1BR / 2BR etc.
facilities	TEXT	Parking, gym, pool, etc.
listed_at	TIMESTAMP	Date listing was recorded

HiddenCost — stores additional cost breakdown per property

© UMHackathon 2026

Email: umhackathon@um.edu.my

Field	Type	Description
cost_id	PK	Unique cost record
property_id	FK	Links to Property
cost_type	ENUM	Parking / Utilities / Deposit / Internet / Admin
amount	DECIMAL	Monthly estimated amount (RM)

RentBenchmark — stores market benchmark by postcode

Field	Type	Description
benchmark_id	PK	Unique benchmark record
postcode	VARCHAR	Area postcode
room_type	VARCHAR	Unit type
avg_rent	DECIMAL	Average market rent
min_rent	DECIMAL	Lowest recorded rent

© UMHackathon 2026

Email: umhackathon@um.edu.my

max_rent	DECIMAL	Highest recorded rent
source	VARCHAR	Kaggle / LPEPH / iProperty
updated_at	TIMESTAMP	Last data refresh

Review — stores unstructured tenant feedback

Field	Type	Description
review_id	PK	Unique review identifier
property_id	FK	Links to Property
source	VARCHAR	Lowyat / Facebook / Reddit
content	TEXT	Raw review or complaint text
sentiment	ENUM	Positive / Neutral / Negative
issue_tag	VARCHAR	Deposit / Maintenance / Management
scraped_at	TIMESTAMP	Date collected

© UMHackathon 2026

Email: umhackathon@um.edu.my

AnalysisResult — stores AI-generated output per user query

Field	Type	Description
result_id	PK	Unique result record
user_id	FK	Links to User
property_id	FK	Links to Property
verdict	ENUM	Great Deal / Acceptable / Avoid
true_cost	DECIMAL	Calculated all-in monthly cost
risk_score	INT	1–10 risk rating
negotiation_tip	TEXT	AI-generated negotiation script
created_at	TIMESTAMP	Query timestamp

© UMHackathon 2026

Email: umhackathon@um.edu.my

Relationships

- One **User** can have many **AnalysisResults**
- One **Property** can have many **HiddenCosts** and many **Reviews**
- One **Property** links to one **RentBenchmark** via postcode
- One **AnalysisResult** references one **User** and one **Property**

3. *Functional Requirements & Scope*

This section defines the system boundaries and focuses on delivering a high-impact Minimum Viable Product (MVP) for demonstration during UMHackathon 2026. The goal is to showcase the technical feasibility of an AI-powered rental decision assistant.

3.1. *Minimum Viable Product:*

#	*Features*	*Description*
1	*Property Analysis Input*	*Users enter property details (location/postcode, asking rent, monthly income). The system validates inputs and sends them to the backend for processing.*
2	*AI Verdict Engine + Result Dashboard*	*The system analyzes affordability, market pricing, and risk factors to generate a final verdict (GREAT DEAL / ACCEPTABLE / AVOID) and presents it through an interactive dashboard with structured visual cards (cost breakdown, risk radar, negotiation tips).*
3	*True Cost Calculator*	*Calculates hidden rental costs (deposit, utilities, fees) and presents a clear monthly breakdown for better financial understanding.*
4	*Risk Radar*	*Integrates keyword-based detection with weighted scoring to produce a quantified risk score and highlight the most critical issue.*
5	*Property Comparison Mode*	*Users can compare up to 3 properties side-by-side with verdicts, pricing benchmarks, and risk insights.*
6	*Market Benchmark Comparison*	*Compares asking rent against postcode average using dataset and returns above/at/below market with percentage difference.*

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*System should handle multiple concurrent users during demo without performance degradation.*	*Deploy backend using scalable platforms and lightweight APIs.*
Reliability	*Analysis requests should consistently return complete results without system crashes.*	*Backend validation, structured JSON responses, and fallback cached AI responses.*
Maintainability	*System components (frontend, backend, AI module, financial engine) must be modular and independently maintainable.*	*Clear folder structure, service-based architecture, and separation of concerns.*
Token Latency	*AI (GLM) responses should return within acceptable time (<5 seconds typical).*	*API timeout handling, cached responses, and loading skeleton UI for users.*
Cost Efficiency	*AI usage should remain within budget constraints during demo.*	*Fixed prompt structure, response caching, and limiting repeated API calls.*
Usability	*Interface must be intuitive and visually clear for demo judges.*	*Flutter UI with structured cards (verdict, cost, risk, negotiation tips).*
Performance	*System should return full analysis within 5–8 seconds*	*Async API calls, caching, and optimized prompt design*
Availability	*System should remain usable even if AI API fails*	*Cached responses + fallback demo data*

3.3. Out of Scope / Future Enhancements

The following features are part of the long-term vision but will not be implemented during UMHackathon 2026:

a. *User authentication & personalized dashboards (saved history of analysed properties)*

b. *Real-time integration with live property listing platforms (e.g., rental APIs)*

c. *Advanced AI negotiation automation (auto-generating messages to landlords)*

d. *Interactive map-based property exploration*

e. *Machine learning model trained on real tenant review datasets instead of mock data*

f. *Long-term financial planning (e.g., buy vs rent simulation with loan calculations)*

g. *Real-time AI streaming responses (token-by-token explanation)*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

For the AI Rent Advisor system, new features—especially those involving the GLM AI module are deployed using a grayscale rollout strategy to minimize risk.

Initially, new features (e.g., updated AI prompts such as improved risk narrative or negotiation coach) are released to 5% of users via backend-controlled feature flags. During this phase, the team monitors:

- *API latency (GLM response time)*
- *Error rates from /analyse endpoint*
- *Consistency of AI-generated outputs (verdict correctness)*

If no anomalies are detected, the rollout is gradually increased (25% → 50% → 100%).
After full rollout, A/B testing is conducted for optimization:

Example: Two versions of negotiation coach prompt

- *Version A: More direct advice*
- *Version B: More conversational Malaysian tone*

Metrics tracked:

- *User engagement (time spent on result screen)*
- *Interaction with negotiation section*

© UMHackathon 2026

- *User feedback (if implemented)*

The better-performing variant is retained and standardized in ai_engine.js.

4.1.2. Emergency Rollback (ER) & Golden Release:

To ensure system reliability, the project implements an Emergency Rollback (ER) strategy.

If critical anomalies are detected:

- *High failure rate in GLM API*
- *Backend endpoint instability*
- *Incorrect verdict generation*

The system automatically triggers ER via the deployment pipeline (Vercel + Railway).

A Golden Release version (last stable version of:

- *ai_engine.js*
- *backend services*
- *prompt configurations*

is redeployed immediately.

Additionally:
cache_service.js provides fallback responses if GLM exceeds 8 seconds, ensuring the demo remains functional even during API instability.

4.1.3. Priority Matrix:

4.1.3.1. The Priority Matrix defines severity levels, trigger conditions, and actions:

Priority	*Trigger condition*	*Action*
P1 – Critical	*GLM API failure rate >10%, or /analyse endpoint returning errors for >5% of requests*	*Trigger Emergency Rollback to Golden Release; activate demo cached responses*
P1 – Critical	*Frontend completely blank or crashing on result screen for any user*	*Immediate hotfix deploy; rollback if unresolved in 15 min*
P2 – High	*GLM API response latency >8 seconds*	*Activate Member 4's cached GLM fallback; alert team; no rollback yet*

© UMHackathon 2026
Email: umhackathon@um.edu.my

P2 – High	*Postcode benchmark lookup returning null for >20% of valid KL postcodes*	*Backend hotfix to dataset lookup; fallback to national average*
P3 – Medium	*Risk Radar score calculation producing out-of-range values (outside 1–10)*	*Log and default to score = 5 with a "data unavailable" flag; no rollback*
P3 – Medium	*Affordability Stress Test returning wrong labels (e.g., Safe when income ratio is dangerous)*	*Log error; flag result to user as "review recommended"; patch in next deploy*
P4 – Low	*Static rent comparison screen rendering with visual misalignment*	*Log; fix in next scheduled release*

*4.2. **Monitoring**:*

4.2.1. The AI Rent Advisor system implements a multi-layer monitoring strategy across backend services, AI processing, frontend stability, and data integrity. Monitoring ensures early detection of anomalies and triggers automated or manual recovery actions.

Component	*What to monitor*	*Tool/Method*	*Alert Trigger*	*Action*
Backend API	*/analyse & /compare endpoint health*	*Railway logs + uptime monitoring*	*>3 consecutive failures OR >5% error rate*	*Page on-call; escalate to P1 if sustained*
AI Layer	*GLM API response time*	*Timestamp logging in ai_engine.js*	*Latency > 8 seconds*	*Activate cache_service.js fallback responses*
	GLM API errors / rate limits	*Try-catch + error logging in ai_engine.js*	*Any 429 or 5xx response*	*Switch to cached responses; alert Member 1*
Frontend	*App crash / blank screen*	*Vercel logs + Flutter error boundary*	*Unhandled exception on result/compare screen*	*Trigger rollback if crash rate >1%*
Data Layer	*Dataset benchmark lookup*	*Logging in pricing_service.js*	*Null return rate >20% (postcode lookup failure)*	*Alert Member 3; fallback to national average*

© UMHackathon 2026

Risk Engine	*Risk score validity*	*Validation in financial_risk_engine.js*	*Score <1 or >10*	*Log issue + override to neutral score (5)*
Integration	*End-to-end system health*	*Daily integration tests (Member 4)*	*Any test failure*	*Notify team; block new merges*

4.3. Assumptions:

The following assumptions are made during development and deployment:

- *Users have a stable internet connection when submitting property input forms and receiving analysis results.*
- *The Z.ai GLM API is available and responsive within 8 seconds under normal load; the team's usage stays within the API's free/allocated rate limits during development and demo.*
- *The mock KL and Selangor rental dataset is sufficiently representative of current Kuala Lumpur and Selangor rental market pricing for postcode-level benchmarking.*
- *The Flutter frontend is deployed as a web app (not native mobile) for demo purposes, accessible via modern browsers.*
- *Demo users will submit realistic property inputs (valid KL or Selangor postcodes, reasonable rent and income figures); extreme or malicious inputs are out of scope for this phase.*
- *Firebase (backend) and Flutter (frontend) remain available and within their free tier limits throughout the project demo period.*
- *The mock complaint dataset used by Member 5's Risk Radar is sufficiently representative of common rental red flags in the Malaysian market.*
- *Cached responses are sufficient for fallback scenarios*

4.4. External Dependencies:

Tools	*Purpose*	*Risks*
Z.ai GLM API	*Core AI engine for verdicts, explanations, and negotiation advice*	*High – API latency, rate limits (429), or service outages (5xx) can block AI responses and break core features; variability in output quality may also affect consistency → mitigated via caching, fallback responses, and prompt validation*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Mock KL & Selangor Rental Dataset	*Provides postcode-based rental benchmarks*	*Medium – data is scraped and may contain inconsistencies (duplicate listings, missing values, or inaccurate prices); limited to KL/Selangor coverage, reducing generalizability; may become outdated over time → mitigated via data cleaning, aggregation (averages), and fallback to default benchmarks*
Firebase Hosting	*Frontend deployment*	*Low – while highly reliable, potential risks include CDN caching delays causing outdated UI to be served after updates; mitigated by cache invalidation and versioned deployments*
Firebase	*Backend hosting*	*Medium – cold start latency and scaling delays may impact API response time under sudden traffic spikes; service misconfiguration or region outage can temporarily disrupt /analyse and /compare endpoints*
Flutter Framework	*Frontend UI rendering*	*Low – rendering inconsistencies or package compatibility issues may affect UI behavior across browsers/devices → mitigated through testing and stable package versions*
Node.js / Python Backend	*Server logic and API handling*	*Low – bugs, memory leaks, or unhandled exceptions may cause endpoint failures or incorrect responses → mitigated via error handling, validation, and testing*

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day 1–2: Project Planning & Requirement Analysis

- *Define project scope and objectives*
- *Identify key features (property search, comparison, rental insights)*
- *Assign team roles and responsibilities*

Day 3–4: System Design

© UMHackathon 2026

Email: umhackathon@um.edu.my

- *Design system architecture (frontend, backend, database)*
- *Create wireframes for main screens (search, results, comparison)*
- *Define database schema (property listings, user preferences, pricing data)*

Day 5–7: Development Phase

- *Develop frontend interface for property search and display*
- *Build backend APIs for data retrieval, filtering, and comparison*
- *Implement basic recommendation or insight logic*

Day 8: Integration & Testing

- *Integrate frontend, backend, and AI components*
- *Perform functional testing (search, filtering, recommendations)*
- *Fix bugs and ensure smooth data flow*

Day 9: Refinement & Optimization

- *Improve UI/UX for better user experience*
- *Optimize performance and response time*
- *Validate accuracy of rental insights*

Day 10: Final Review & Preparation

- *Final system testing and debugging*
- *Prepare demo and presentation materials*

5.2. ***Team Members Role:***

Member 1 — AI Integration Lead

Responsible for developing and integrating intelligent features such as rental price insights, recommendations, or prediction models to support user decision-making with Z.ai GLM.

Member 2 — Frontend Developer

Designs and implements the user interface, including search functionality, property listings, and comparison views, ensuring a smooth and user-friendly experience.

Member 3 — Backend 1

Develops core backend logic, including API development, data handling, and database schema design for property and user data.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Member 4 — Backend 2 (Integration)

Handles system integration between frontend, backend, and AI components, ensuring smooth communication and consistent data flow across the system.

Member 5 — Financial & Risk Engineer

Analyzes rental affordability, pricing trends, and potential financial risks, contributing features such as budget recommendations and cost comparison insights.

5.3. Recommendations:

- ***Optimize database performance***
  Use indexing on key fields (location, price) to speed up search and filtering
- ***Adopt modular architecture***
  Separate frontend, backend, and AI components for easier scaling and maintenance
- ***Use cloud deployment (eg: Google Run Cloud)***
  Ensure better scalability, availability, and system reliability
- ***Ensure data validation & error handling***
  Improve system stability and prevent incorrect or incomplete user inputs
- ***Design for scalability***
  Structure the system to support future features such as AI predictions and larger datasets

© UMHackathon 2026